拼接path

```
const testCase = [
    {
        path: '/jobs',
        routes: [
            {
                path: '/auto_create',
            },
            {
                path: '/:jobId',
                routes: [
                    {
                        path: '/info',
                    },
                    {
                        path: '/setting',
                    },
                    {
                        path: '/manager',
                    },
                    {
                        path: '/source',
                    },
                    {
                        path: ['/smart_recruitment'],
                    },
                    {
                        path: '/satisfaction',
                        routes: [
                            {
                                path: ['/pandect/:candidateId', '/application/:applicationId', '/:id'],
                            },
                        ],
                    },
                ],
            },
        ],
    },
];
```

```

// 得到结果
// [
//   '/jobs/auto_create',
//   '/jobs/:jobId/info',
//   '/jobs/:jobId/setting',
//   '/jobs/:jobId/manager',
//   '/jobs/:jobId/source',
//   '/jobs/:jobId/smart_recruitment',
//   '/jobs/:jobId/satisfaction/pandect/:candidateId',
//   '/jobs/:jobId/satisfaction/application/:applicationId',
//   '/jobs/:jobId/satisfaction/:id',
// ];

const exchangeArr = (arr) => {
    let resArr = [];
    const fn = (arr, path = "") => {
        let newPath;
        for (let o of arr) {
            if (Array.isArray(o.path)) {  //数组:
                newPath = [];
                for (let pathInner of o.path) {
                    newPath.push(path + pathInner)
                }
            } else {  //字符串: 直接拼接
                newPath = path + o.path;  //拼接path
            }

            if (o.routes && o.routes.length) {  //如果有子集, 继续拼接
                if (Array.isArray(newPath)) {
                    for (let pathItem of newPath) {
                        fn(o.routes, pathItem);
                    }
                } else {
                    fn(o.routes, newPath);
                }
            } else {  //如果没有子集, 则返回
                if (Array.isArray(newPath)) {
                    resArr = [...resArr, ...newPath];
                } else {
                    resArr.push(newPath);
                }
```

```
            }
        }
    }
    fn(arr);
    return resArr;
}
console.log(exchangeArr(testCase));
```

转化对象输出

```
//输入: getJson("a.b.c", 3);
//输出
// obj = {
//     a: {
//         b: {
//             c: 3
//         }
//     }
// };

function getJson(keys, value) {
    const keysArr = keys.split(".").reverse();
    let obj = {
        [keysArr[keysArr.length - 1]]: value
    };
    for (let i = 1; i <= keysArr.length - 1; i++) {
        const key = keysArr[i];
        obj = {
            [key]: obj
        };
    }
    return obj;
}

function getJson(keys, value) {
    const keysArr = keys.split(".").reverse();
    return keysArr.reduce((did, key, index) => {
        if (index === 0) {
            did = {
```

```
                [key]: value
            };
        } else {
            did = {
                [key]: did
            };
        }
        return did;
    }, {});
}


console.log(getJson2("a.b.c", 3));
```

this 笔试题

```
// 严格模式下，局部（函数）作用域中的 this 默认指向 undefined（全局的 this 仍然指向 window）
"use strict";
console.log(this);  //window
{
  console.log(this);  //window 块级作用域不受影响
}

function test() {
      console.log(this);  //undefined
}

test();
```

```
var a = 1;

var obj = {
    a: 2,
    test() {
        return function () {
            console.log(this.a);
        };
```

```
    },
    test2: () => {
        return function () {
            console.log(this.a);
        };
    },
    test3: () => {
        return () => {
            console.log(this.a);
        };
    },
    test4() {
        return () => {
            console.log(this.a);
        };
    },
};
const obj2 = { a: 3 };
obj2.test = obj.test();
obj2.test2 = obj.test2();
obj2.test3 = obj.test3();
obj2.test4 = obj.test4();
obj2.test();
obj2.test2();
obj2.test3();
obj2.test4();
```

```
// class 中默认严格模式
// 严格模式下，局部（函数）作用域中的 this 默认指向 undefined（全局的 this 仍然指向 window）
class Person {
  fn() {
    console.log(this);
  }
  static fn2() {
    console.log(this);
  }
}

let p = new Person();
```

```
let fn = p.fn;
let fn2 = Person.fn2;
fn();
fn2();
p.fn();
Person.fn2();
```

```
function res() {
  let as = () => {
    console.log(this);
  };
  as();
}

let obj = {
  name: "sss"
};

res();
res.call(obj);
```

```
var msg1 = 1;
const msg2 = 2;
const obj = {
  msg1: 3,
  msg2: 4,
  log1: function() {
    console.log(this.msg1, this.msg2);
  },
  log2: () => {
    console.log(this.msg1, this.msg2);
  }
};

const { log1, log2 } = obj;
obj.log1();
obj.log2();
```

```
log1();
log2();
```

```
var name = "aaa";

class Person {
    constructor() {
        this.name = "sss";
    }

    static say = () => {
        console.log(this.name);
    };
    see = () => {
        console.log(this.name);
    };
}

function cry() {
    console.log(this.name);
}

Person.say();
new Person().see();
cry.call(Person);
```

其他笔试题

```
log(x);
log(y);
log(z);
log(m);
log(n);
let z = 0;
var x = y = z++;
if (x) {
  let m = 3;
```

```
  var n = 4;
}
log(x);
log(y);
log(z);
log(m);
log(n);

var log = function(v) {
  try {
    console.log(v);
  } catch (e) {
    console.log("error");
  }
};

//答案: log is not a function
```

宏任务微任务

```
console.log(1);

setTimeout(() => {
  console.log(2);
}, 0);

Promise.resolve(3).then(res => {
  console.log(res);
}).then(res => {
  return Promise.resolve(4);
}).finally(function(res) {
  console.log(res);
}).then(res => {
  console.log(res);
});

Promise.resolve(5).then(res => console.log(res));

```

```
//1 3 5 undefined 4 2
```

```
setTimeout(() => {
    console.log(1);
    Promise.resolve().then(() => {
        console.log(2);
    });
}, 0);

setTimeout(() => {
    console.log(3);
    Promise.resolve().then(() => {
        console.log(4);
    });
}, 0);
//1 2 3 4
```

```
async function async1() {
  console.log("2 - async1 start");
  await async2();
  console.log("4 - async1 end");
}

async function async2() {
  console.log("3 - async2");
}

console.log("1 - script start");

setTimeout(function() {
  console.log("8 - setTimeout");
});

async1();

new Promise(function(resolve) {
  console.log("5 - promise1");
```

```
  resolve();
}).then(function() {
  console.log("7 - promise2");
});

console.log("6 - script end");

// 1 - script start
// 2 - async1 start
// 3 - async2
// 5 - promise1
// 6 - script end
// 4 - async1 end
// 7 - promise2
// 8 - setTimeout
```

```
async function fn1() {
  console.log("1");
}

async function fn2() {
  console.log("2");
}

async function result() {
  await fn1();
  fn2();
  console.log("3");
}

result();
console.log(4);

//1 4 2 3
```

```
async function fn1() {
  console.log(1);
```

```
}

async function fn2() {
  console.log(2);
}

async function result() {
  await fn1();
  fn2();
  setTimeout(() => console.log(3));
  Promise.resolve().then(res => console.log(4));
  console.log(5);
}

result();
console.log(6);
Promise.resolve().then(res => console.log(7));
setTimeout(() => console.log(8));

// 1 6 2 5 7 4 8 3
```

创建分页组件

```
/**
 * 创建分页组件需要的按钮数组
 * @param total    总条数
 * @param size     每页几条
 * @param page     当前第几页
 * @param btnCount 最多看到几个按钮
 */
function f(total, size, page, btnCount = 5) {
  const allLength = Math.ceil(total / size);
  const lenArr = Array.from({ length: allLength }, (item, index) => index + 1);
  if (allLength < btnCount) return lenArr;
  let targetArr = [];
  if (page < Math.ceil(btnCount / 2)) {
    targetArr = Array.from({ length: btnCount }, (item, index) => index + 1);
  } else if (page > allLength - Math.floor(btnCount / 2)) {
    targetArr = lenArr.slice(-btnCount);
```

```
  } else {
    targetArr = lenArr.slice(page - Math.ceil(btnCount / 2) < 0 ? 0 : page - Math.ceil(btnCount / 2), page + Math.floor(btnCount / 2));
  }
  return targetArr;
}

console.log(f(100, 10, 1, 5));  //[ 1, 2, 3, 4, 5 ]
console.log(f(100, 10, 7, 5));  //[ 5, 6, 7, 8, 9 ]
console.log(f(100, 10, 5, 5));  //[ 3, 4, 5, 6, 7 ]
console.log(f(300, 10, 10, 6));  //[ 8, 9, 10, 11, 12, 13 ]
```

手写map

- 用 callbackFn 执行数据中的每一个元素，将返回结果组成新数组返还
- callbackFn 中别忘了传入 i 和 this

```
Array.prototype.map2 = function(callbackFn, thisArg) {
  if (this === null || this === undefined) {
    throw new TypeError("Cannot read property 'map' of null or undefined");
  }
  if (typeof callbackFn !== "function") {
    throw new TypeError(callbackFn + " is not a function");
  }

  let length = this.length;
  let resArr = [];
  for (let i = 0; i < length; i++) {
    resArr[i] = callbackFn.call(thisArg, this[i], i, this);
  }
  return resArr;
};
```

意外发现

```
✨ 手写map意外发现
let arr = [0, []];
let arr2 = arr.map(item => item);
```

```

arr.push(1);
console.log(arr2[2]);  //undefined

arr[1].push(1);
console.log(arr2[1]);  //[1]

✨ 结论：map 执行原理：将传入的 arr 元素执行回调函数后放进 newArr 中返回，如果传入的 arr 中存在引用类型的元素，
可能也会存在于 newArr 中，arr 和 newArr 中的引用类型元素之间，可能会存在关联
```

手写push

- **可以一次性push多个元素**
- **最后返还push后数组的长度**

```
Array.prototype.push2 = function(...item) {
  let length = item.length;
  for (let i = 0; i < length; i++) {
    this[this.length] = item[i];
  }
  return this.length;  //注意最后要返回操作后的数组长度
};
```

手写pop

```
Array.prototype.pop2 = function() {
  let length = this.length;
  if (length === 0) {
    return undefined;
  }
  length--;
  const value = this[length];
  this.length = length;
  return value;
};
```

手写filter

- **callbackFn 中别忘了传入 i 和 this**

```
Array.prototype.filter2 = function(callbackFn, thisArg) {
  if (this === null || this === undefined) {
    throw new TypeError("Cannot read property 'filter' of undefined or null");
  }
  if (typeof callbackFn !== "function") {
    throw new TypeError(callbackFn + "is not a function");
  }

  let length = this.length;
  let resArr = [];
  for (let i = 0; i < length; i++) {
    if (callbackFn.call(thisArg, this[i], i, this)) {
      resArr[resArr.length] = this[i];
    }
  }
  return resArr;
};
```

手写reduce

```
Array.prototype.reduce2 = function(callbackFn, initialValue) {
  if (this === null || this === undefined) {
    throw new TypeError("Cannot read property 'map' of null or undefined");
  }
  if (typeof callbackFn !== "function") {
    throw new TypeError(callbackFn + " is not a function");
  }

  let len = this.length;
  let did = initialValue;
  let i = 0;
  if (did === undefined) {
    if (len === 0) {
      throw new TypeError("Reduce of empty array with no initial value");
    }
```

```

    for (; i < len; i++) {
      if (i in this) {
        did = this[i];
        i++;
        break;
      }
    }
  }

  for (; i < len; i++) {
    if (i in this) {
      did = callbackFn(did, this[i], i, this);
    }
  }
  return did;
};
```

清除浮动的方式

```
.parent{
    overflow:hidden
}
或者:
.parent{
    ::after{
        display:block;
        content:"";
        clear:both;
    }
}
```